1ère année, IUT réseau & télécommunications

Fondamentaux de la Programmation

Nils Laurent
nils.laurent@univ-st-etienne.fr

Mode de fonctionnement

Dans les grandes lignes

- arriver après l'appel $\Longrightarrow$ retard
- jeu en cours $\Longrightarrow$ exclusion du cours
- le silence est indispensable

L’intelligence artificielle

Modèles de langages classiques aujourd’hui

- entrée : prompt (ce que vous écrivez)
- sortie : séquence la plus probable (statistiquement)

Choix implicite

+ rapidité d’écriture du code
+ probabilité d’erreur plutôt faible (en fonction du problème)
- consommation en gigawatts
- une réponse potentiellement non-neutre (détenteurs de grok, deepseek, etc.)
- transmission de l’information du prompt (dès la saisie)

L'intelligence artificielle

Règles (pour ce cours)

- IA interdite aux examens
- Transparence sur son utilisation

En TD/TP

- fortement déconseillée (l'objectif est d'obtenir une certaine maitrise)
- il ne sera jamais demandé de l'utiliser

Remarque : pour résoudre les problèmes, il est demandé d'utiliser uniquement ce qui a été vu en cours.

Structure du cours

1. Algorithmes + langage Python
2. Langage C
3. Annexe : fonctions utilitaires
4. Introduction à git

Chapitre 1 : Les algorithmes et leur écriture en langage Python

Les algorithmes

Définition

Algorithme Ensemble de règles opératoires dont l'application permet de résoudre un problème énoncé au moyen d'un nombre fini d'opérations.

Existait bien avant les ordinateurs : travaux du mathématicien persan Al-Khwârizmî (900).

Applications : presque partout

Par exemple, pour les **réseaux et télécommunications**

- Protocoles : HTTP, DNS, TCP, IPv4, etc.
- Systèmes d'exploitation Cisco (Routeurs, Switch, etc.)
- Logiciels drivers : cartes physiques d'interface réseaux

Les fonctions : prototype d'une fonction

Définition

Prototype déclaration d'une fonction. Elle contient le nombre et le type des paramètres et le type de la valeur de retour.

Syntaxe (cas avec deux paramètres/arguments)

```
def nom_de_fonction(argument1: TYPE1, argument2: TYPE2) -> TYPE_DE_SORTIE:
```

Exemple (nous verrons plus tard ce que signifie `int`)

```
def somme(a: int, b: int, c: int) -> int:
```

Les commentaires

Un commentaire n'a aucun impacte sur l'algorithme. C'est un texte qui aide le lecteur à comprendre ce qui est écrit.

Syntaxe
commence par le symbole #

```
# ceci est un commentaire
```

Corps d'une fonction et indentation

Définition

Indentation Espacement à gauche sur chaque ligne de code.

Le corps d'une fonction, c'est-à-dire le code propre à celle-ci, doit **obligatoirement** être indenté. Cela signifie qu'il faut ajouter un espacement sur la gauche de toutes les lignes qui constituent la fonction.

```
def ma_fonction():
    # ligne indentée, donc dans le corps
    # ligne indentée, donc dans le corps

# ligne non-indentée, elle ne fait pas partie du corps de la fonction

    # à cause de la ligne précédente, ne fait pas partie du corps de la fonction
```

Afficher à l'écran

Créer un fichier python

```
hello_world.py
```

Python fonctionne avec une console, on peut y afficher du texte.
Dans le fichier, écrire le code suivant.

```
print("Hello world") # affichage du texte sur l'ecran
```

Dans un terminal

```
cd chemin/vers/le/fichier
python hello_world.py
```

Spécification

Elle décrit le « quoi », pas le « comment »

- **entrée** :
  élément variable de l'énoncé
- **pre-condition** :
  condition sur les paramètres d'entrée
- **sortie** :
  élément de la réponse au problème
- **post-condition** :
  condition sur les paramètres de sortie

Spécification de la fonction "ecart"

```
def ecart(a: float, b: float) -> float:
    """
    :pre-cond: a > 0
    :pre-cond: b > a
    :sortie e: nombre à virgule (float)
    :post-cond: écart entre a et b
    """
```

Convention de codage

Nous écrirons toujours le code dans des fonctions.

```
def main():
    print("Hello world")
```

A la toute fin du fichier, nous aurons toujours le code suivant

```
if __name__ == "__main__":
    main()  # appel de la fonction main
```

Ce qui s'affiche

```
Hello world
```

Des détails sur l'affichage

Nous écrirons toujours le code dans des fonctions.

```
def main():
    print("Hello world")
    print("Ligne 2")
    print("Ceci est", "la", end="")
    print("ligne", 3)

if __name__ == "__main__":
    main()  # appel de la fonction main
```

Ce qui s'affiche

```
Hello world
Ligne 2
Ceci est la ligne 3
```

Les variables

Une **variable** est désignée par un **identifiant** et peut **mémoriser** une valeur d'un certain **type**. La valeur de la variable peut changer dans le temps.

Syntaxe sans typage

```
identifiant = valeur
```

Syntaxe avec typage

```
identifiant: type = valeur
```

Remarque : les paramètres de fonction sont des variables.

Types des variables

Exemples (avec typage)

```
def exemple_variables():
    nb_eleves: int = 30         # nombre d'élèves dans la classe
    moyenne_age: float = 19.5   # age moyen de la classe
    id_classe: str = "A1"       # identifiant de la classe
    groupe_td: bool = True      # vrai, c'est un groupe TD
    anglais: bool = False       # faux, le cours est en français
```

L'identifiant commence par une lettre (ou par _)

Les variables

Dans le tableau, quels sont les identifiants ?
Répondre par vrai ou faux.

Identifiant	Vrai/Faux
id50	
2eme	
Groupe1	
"Groupe1"	

Les variables

Dans le tableau, quels sont les identifiants ?
Répondre par vrai ou faux.

Identifiant	Vrai/Faux
id50	Vrai
2eme	Faux
Groupe1	Vrai
"Groupe1"	Faux

Affectation à une variable

Exemples (sans typage)

```
def exemple_variables():
    x = 30  # 30 est affecté à "x"
    y = 10  # 10 est affecté à "y"
    x = y   # la valeur de "y" est affecté à "x"
    # maintenant "x" vaut 10
```

Opérations sur des variables

Soient les variables a et b

```
def operations(a: float, b: float):
    a + b   # addition
    a - b   # "a" moins "b"
    a * b   # produit
    a / b   # "a" divisé par "b"
    a // b  # division entière
    a ** b  # "a" puissance "b"
    a % b   # reste de "a" divisé par "b"
```

Exemple : si `a = 16` et `b = 10` alors `a // b` est égal à 1.

Exemple d'algorithme

```
def calcul_ecart(a: float, b: float) -> float:
    """
    :pre-cond: a > 0
    :pre-cond: b > a
    :sortie ecart: float
    :post-cond: écart entre a et b
    """
    ecart = b - a
    return ecart  # cette instruction renvoie la sortie
```

Appel de fonction

```python
def calcul_ecart(a: float, b: float) -> float:
    """
    :pre-cond: a > 0
    :pre-cond: b > a
    :sortie e: float
    :post-cond: écart entre a et b
    """
    e = b - a
    return e  # cette instruction renvoie la sortie

def main():
    resultat = calcul_ecart(3, 10)
    print("le résultat est", resultat)

if __name__ == "__main__":
    main()
```

Retour de fonction et affichage sur l'écran

Retour de fonction :

```
def ma_fonction(a: float, b: float) -> float:
    return a + b  # renvoie a + b

def main():
    resultat = ma_fonction(3, 10)
```

Affichage sur l'écran :

```
def ma_fonction(a: float, b: float) -> None:
    print(a + b)  # affichage de a + b

def main():
    ma_fonction(3, 10)
```

Un autre exemple

Cet algorithme calcul $a^2 + 2ab + b^2$ et l'affecte à y
Le mot clé **return** est utilisé pour renvoyer le résultat y

```python
def ma_fonction(a: float, b: float) -> float:
    y = a ** 2 + 2 * a * b + b ** 2
    return y

def main():
    resultat = ma_fonction(3, 10)
    print("a = ", 3, "b = ", 10, "(a + b)**2 = ", resultat)

if __name__ == "__main__":
    main()
```

Renvoyer plusieurs résultats

Après `return` lister les variables en séparant par des virgules

```
def plusieurs_sorties(a: float, b: float) -> tuple[float, float]:
    """
    :sortie m: float
    :sortie s: float
    :post-cond: m est la moyenne de a et b
    :post-cond: s est la somme de a et b
    """
    s = a + b
    m = s / 2
    return m, s
```

Exercice 1

```
def echanger(a: float, b: float) -> tuple[float, float]:
    """
    :sortie a: float
    :sortie b: float
    :post-cond: les valeurs de a et b sont échangées
    """
    pass
```

```python
def echanger(a: float, b: float) -> tuple[float, float]:
    """
    :sortie a: float
    :sortie b: float
    :post-cond: les valeurs de a et b sont échangées
    """
    c = a
    a = b
    b = c
    return a, b
```

Expression booléennes

Les expressions booléennes prennent la valeur vrai/faux

```
def expression_booleennes(a: float, b: float, x: bool, y: bool):
    a > b     # strictement supérieur
    a >= b    # supérieur ou égale
    a < b     # strictement inférieur
    a <= b    # inférieur ou égale
    a == b    # égalité
    a != b    # différent de
    x and y   # "et" logique
    x or y    # "ou" logique
    not x     # "non" logique
```

Chaque ligne vaut `True` ou `False`

Exemple

```
def est_plus_grand(a: float, b: float) -> bool:
    """
    :sortie c: bool
    :post-cond: "c" contient vrai si a > b
    """
    c = a > b  # "c" contient forcément "True" ou "False"
    return c
```

Exercice 2

Construire `a >= b` en utilisant les autres expression booléennes.

Construire `a >= b` en utilisant les autres expression booléennes.

Exemples :

- `(a > b) or (a == b)`
- `not (a < b)`

Les conditions

Cas général

```
def ma_fonction():
    # ...
    if EXPRESSION_BOOLEENNE_1:
        BLOC1_LIGNE_1
        etc.
    elif EXPRESSION_BOOLENNE_2:
        BLOC2_LIGNE_1
        etc.
    else:
        BLOC3_LIGNE_1
        etc.
    # suite du code ...
```

Exemple :

```
def positif(a: float) -> str:
    txt: str = "négatif"
    if a >= 0:
        txt = "positif"
    return txt
```

Il faut indenter sous **if**, **elif** et **else**.

Exemple

```
def feu_de_circulation(f: str) -> str:
    txt = ""
    if f == "v":
        txt = "Le feu est vert"
    elif f == "o":  # cette instruction est un "sinon si"
        txt = "Le feu est orange"
    else:  # cette instruction est un "sinon"
        txt = "Le feu est rouge"
    return txt
```

Quelle différence ? (A)

```
def ma_fonction():
    x = -1
    c = 0
    if x != 0:
        c = c + 1
    if x != 1:
        c = c + 1
    else:
        c = c + 1
    return c
```

```
def ma_fonction():
    x = -1
    c = 0
    if x != 0:
        c = c + 1
    elif x != 1:
        c = c + 1
    return c
```

Quelle différence ? (B)

```python
def ma_fonction():
    x = 1
    c = 0
    if x == 0:
        c = c + 1
        c = c + 1
    return c
```

```python
def ma_fonction():
    x = 1
    c = 0
    if x == 0:
        c = c + 1
    c = c + 1
    return c
```

Exercice 3

```
def valeur_absolue(x: float) -> tuple[float, int]:
    """
    :sortie y: float
    :sortie signe: int
    :post-cond y: valeur absolue de x
    :post-cond signe: signe de x
    """
```

```python
def valeur_absolue(x: float) -> tuple[float, int]:
    """
    :sortie y: float
    :sortie signe: int
    :post-cond: y est la valeur absolue de x
    :post-cond: signe est le signe de x
    """
    if x >= 0:
        y = x
        signe = 1
    else:
        y = -x
        signe = -1
    return y, signe
```

Les boucles

Une **boucle** permet de répéter une séquence d'instructions un certain nombre de fois.

Il y en a deux qui sont très courantes :

- La boucle **while**
- La boucle **for**

La boucle while

Cas général

```
def ma_fonction():
    # ...
    while EXPRESSION_BOOLEENNE:
        # bloc de code à répéter
        BLOC_LIGNE_1
        BLOC_LIGNE_2
        etc.
    # suite du code ...
```

Exemple

```
def somme(n: int) -> int:
    """
    :pre-cond: "n" est positif
    :sortie s: int
    :post-cond s: somme sur [1, n]
    """
    s = 0
    i = 0
    while i <= n:
        s = s + i
        i = i + 1
    return s
```

Exercice 4 : trace de l'exécution

On peut tracer l'exécution avec un tableau :

- une colonne pour la ligne concernée
- autant de colonnes que nécessaires pour décrire l'état des variables

Compléter le tableau pour $n = 2$

L. tab	L. algo	s	i
1	1	0	?
2	2		
3	3		

```python
def somme(n: int) -> int:
    """
    :pre-cond: n > 0
    :sortie s: int
    :post-cond s: somme sur [1, n]
    """
    s = 0           # ligne 1
    i = 1           # ligne 2
    while i <= n:   # ligne 3
        s = s + i   # ligne 4
        i = i + 1   # ligne 5
    return s        # ligne 6
```

L. tab	L. algo	s	i
1	1	0	?
2	2	0	1
3	3	0	1
4	4	1	1
5	5	1	2
6	3	1	2
7	4	3	2
8	5	3	3
9	3	3	3
10	6	3	3

```python
def somme(n: int) -> int:
    """
    :pre-cond: n > 0
    :sortie s: int
    :post-cond s: somme sur [1, n]
    """
    s = 0           # ligne 1
    i = 1           # ligne 2
    while i <= n:   # ligne 3
        s = s + i   # ligne 4
        i = i + 1   # ligne 5
    return s        # ligne 6
```

La boucle for

Cas général

```
def ma_fonction():
    # ...
    for VARIABLE_ITER in ITERABLE:
        # bloc de code à répéter
        BLOC_LIGNE_1
        BLOC_LIGNE_2
        etc.
    # suite du code ...
```

Exemple

```
def somme(n: int) -> int:
    """
    :pre-cond: n > 0
    :sortie s: int
    :post-cond s: somme sur [1, n]
    """
    s: int = 0
    for i in range(0, n):
        s = s + i + 1
    return s
```

Comparaison

Syntaxe python de la boucle **while**

```
def somme_n(n):
    i = 1
    s = 0
    while i <= n:
        s = s + i
        i = i + 1
    return s
```

Syntaxe python de la boucle **for**

```
def somme_n(n):
    s = 0
    for i in range(0, n):
        s = s + i + 1
    return s
```

Les tableaux

Un **tableau** est une liste ordonnée de n valeurs du même type.

Syntaxe

- Ses valeurs sont indiquées entre crochet
- Ses valeurs sont séparées par une virgule
- Le `i`-ème élément existe à l'indice `i-1`
- L'accès se fait également avec les crochets

Exemple

```
def six_nb_premiers() -> int:
    x = [2, 3, 5, 13, 11, 7]
    m = x[3]   # maximum dans x
    return m
```

Comment s'écrit l'accès au premier élément de "liste" ?

Tableaux : autres exemples

Attention, on numérote à partir de zéro

```
def exemple():
    T = [13, 8, 8, 80, -5] # tableau de taille 5
    x_0 = T[0]  # 13
    x_max = T[3]  # 80
    x_min = T[4]  # -5
```

Exemple de création d'un tableau

```
def creer_tableau(n: int) -> list[int]:
    tab = [0] * n # tableau de taille n contenant des zéros
    i = 0
    while i < n:
        tab[i] = i + 1
        i = i + 1
    return tab
```

Exercice 5 : tableaux

```
def somme_tab(tab: list[int], n: int) -> int:
    """
    :pre-condition: tab est de taille n
    :sortie s: somme des éléments de tab
    """
```

```python
def somme_tab(tab: list[int], n: int) -> int:
    """
    :pre-condition: tab est de taille n
    :sortie s: somme des éléments de tab
    """
    s = 0
    i = 0
    while i < n:
        s = s + tab[i]
        i = i + 1
    return s
```

Chaine de caractères

Une chaine de caractères est un tableau

```
e = "abcdef" # une chaine de caracteres est un tableau
f = e[1] # c'est la lettre 'b'
```

On peut assembler du texte : opérateur de **concaténation**

```
g = "abcdef" + "ghijkl" # maintenant g vaut "abcdefghijkl"
```

Procédure ou fonction ?

- Une fonction retourne un résultat
- Une procédure ne retourne pas de résultat

```
def fibonacci_next(a: int, b: int) -> tuple[int, int]:
    return b, a+b
```

```
def ajouter_au_tab(tab: list[int], n: int, a: int):
    i = 0
    while i < n:
        tab[i] = tab[i] + a
        i = i + 1
```

Bonnes pratiques de programmation

- Le choix de nom de variable : simple et porteur de sens
- Les fonctions tiennent à peu près sur l'écran (pas trop longues)
- Respecter l'indentation (vous n'aurez pas le choix avec Python)

Reconnaissez-vous cet algorithme ?

```
def k(i):
    rxkfghh = "gh"
    if i == 1:
        p = 7
        a = p+2
        rxkfgh=1
        rxkfghh="temporaire"
        print(rxkfghh, rxkfgh)
    else:
        rxkfghh2 = "tempoaussi"
        rxkfgh = i*k(i-1)
        print(rxkfghh, rxkfgh)
    return rxkfgh
```

Reconnaissez-vous cet algorithme ?

```
def factorielle(rang):
    if rang == 1:
        resultat = 1
        print("factorielle(", rang, ") = ", resultat)
        return resultat
    else:
        resultat = rang * factorielle(rang - 1)
        print("factorielle(", rang, ") = ", resultat)
        return resultat

if __name__ == "__main__":
    factorielle(3)
```

Sortie console :

```
factorielle( 1 ) =  1
factorielle( 2 ) =  2
factorielle( 3 ) =  6
```

Exercice 6 : tableaux à valeurs uniques

```
def unique(tab: list[int], n: int) -> bool:
    """
    :pre-cond: tab est de taille n
    :sortie s: vrai si tab contient des valeurs uniques, faux sinon
    """
```

Complexité

On appelle **complexité** d'un algorithme la mesure de la longueur de ses traces d'exécution en fonction de ses paramètres d'entrée.

On peut par exemple l'étudier :

- au pire cas ;
- en moyenne.

Complexité

Quel est la complexité de l'algorithme ci-dessous ?

```
def somme_tab(tab: list[int], n: int) -> int:
    """
    :pre-condition: tab est de taille n
    :sortie s: somme des éléments de tab
    """
    s = 0
    i = 0
    while i < n:
        s = s + tab[i]
        i = i + 1
    return s
```

Exécution d'une fonction

```
def factorielle(rang: int) -> int:
    i = 1
    resultat = 1
    while i <= rang:
        resultat = resultat * i
        i = i + 1
    return resultat

def main():
    res = factorielle(2)
    print(res)  # affiche 2
    print(factorielle(3))  # affiche 6
    print(factorielle(4))  # affiche 24

if __name__ == "__main__":
    main()
```

Valeur par défaut

La valeur par défaut d'un argument est donnée par la syntaxe suivante

```
def cout_transaction(prix_unitaire: int, quantite: int = 1):
    return prix_unitaire * quantite
```

Chapitre 2 : Le langage C

Test d'une fonction

Le langage C est compilé

$$\text{Code C} \longrightarrow \text{Compilateur} \longrightarrow \text{Exécutable}$$

Un exécutable est une séquence d'instruction élémentaires que le processeur sais effectuer, mais plus difficile à comprendre pour nous.

Les fonctions

Syntaxe du prototype de fonction

- type de sortie
- nom de fonction
- entre parenthèses le type et nom des arguments

```
int ajouter(int x, int y)
```

Le C est à typage statique : le type des variables est explicite.

Les fonctions

Syntaxe du corps de fonction : entre accolades

```
int ajouter(int x, int y)
{
    // on ajoute x et y
    return x + y;
}
```

Les commentaires sont indiqués par “//”.

La fonction principale

La première fonction du programme à s'exécuter est **main**

```
int main()
{
    // instructions a executer dans le programme
    return 0;
}
```

Les commentaires sont indiqués par "//".

Types des variables

```
int a = -5;         // nombre entier
float b = 0.15;     // simple precision
double c = 0.15;    // double precision
char d = 'z';       // une lettre

// separer la declaration et l'affectation
char e;             // declaration
e = 'y';            // affectation
```

Types des variables

Type str ? : n'est pas un type de base du C ... (on verra ça plus tard)

```
"quel est le type de ce texte ???";
```

Type bool ? : idem, ça n'existe pas, on utilise `int` à la place

- *faux* s'écrit `0`
- *vrai* s'écrit `1`
- toutes les autres valeurs `2, 3, 4, ...` ou `-1, -2, -3, ...` signifient *vrai*

Librairie pour l'affichage

Pour l'affichage, on peut utiliser la librairie **stdio**.
En C, les librairies s'utilisent via la directive **include** :

```
#include <stdio.h>
```

où

- Le symbole # désigne le début d'une directive préprocesseur
- Le fichier stdio.h de la librairie est donnée comme paramètre enter < et >

Librairie pour l'affichage

La librairie contient la commande **printf**.
Elle permet d'afficher une chaine de caractères

```
"exemple de chaine de caracteres"
```

Par exemple :

```
#include <stdio.h>

int main()
{
    printf("Hello world!");

    return 0;
}
```

où **void** indique l'on ne retourne rien.

Librairie pour l'affichage

On peut afficher des variables

```c
#include <stdio.h>

int main()
{
    int a = 10;
    int b = 20;
    printf("a = %i, b = %i\n", a, b);

    return 0;
}
```

Ici, %i permet d'afficher un entier et \n de passer à la ligne.
A chaque %, on donne une variable en argument (ici a et b).

Le type "void" et les procédures

Une procédure ne retourne rien. En C, le type d'une procédure est "void".

```c
void ma_procedure()
{
    printf("Hello world\n");
}
```

Symboles pour l'affichage avec printf

Il faut spécifier dans printf

```
void affichage(int a, float b, double c, char d)
{
    printf("%i \n", a);  // int
    printf("%d \n", a);  // int
    printf("%f \n", b);   // float
    printf("%lf \n", c);   // double
    printf("%c \n", d);   // char
    printf("%s \n", "du texte...");
}
```

Liste exhaustive : https://cplusplus.com/reference/cstdio/printf/

Opérations

Soient les variables a et b

```
void operations(float a, float b)
{
    a + b;   // addition
    a - b;   // "a" moins "b"
    a * b;   // produit
    a / b;   // "a" divisé par "b"
    a % b;   // reste de "a" divisé par "b"
}
```

Exercice 7 : parité

Écrire et utiliser une fonction qui affiche le reste de la division par deux.
Comme pour Python, l'opérateur % permet de récupérer le reste d'une division.

Écrire et utiliser une fonction qui affiche le reste de la division par deux.
Comme pour Python, l'opérateur % permet de récupérer le reste d'une division.

```c
#include <stdio.h>

void parite(int a)
{
    int r = a % 2;
    printf("reste de %i divise par 2 = %i\n", a, r);
}

int main()
{
    parite(3);
    parite(4);
    parite(5);
    return 0;
}
```

Conditions

Opérateur booléens :

égale	==
différent	!=
inf. ou égale	<=
inférieur	<
sup. ou égale	>=
supérieur	>
non logique	!
et logique	&&
ou logique	||

Syntaxe C de la structure de **if**

```c
void feu_de_circulation(char f)
{
    if (f == 'v')
    {
        printf("Le feu est vert\n");
    }
    else if (f == 'o')
    {
        printf("Le feu est orange\n");
    }
    else
    {
        printf("Le feu est rouge\n");
    }
}
```

Exercice 8

Écrire un programme en C qui répond aux conditions suivantes :

```c
double valeur_absolue(double x)
{
    /*
    :sortie y: double
    :post-condition: y est la value absolue de x
    */
}
```

Solution

```
double valeur_absolue(double x)
{
    /*
    :sortie y: double
    :post-condition: y est la value absolue de x
    */
    double y = x;
    if (x < 0)
    {
        y = -x;
    }
    return y;
}
```

Boucles

Syntaxe C de la boucle **while**

```c
int somme_n(int n)
{
    int i = 1;
    int s = 0;
    while (i <= n)
    {
        s = s + i;
        i = i + 1;
    }
    return s;
}
```

Syntaxe C de la boucle **for**

```c
int somme_n(int n)
{
    int s = 0;
    for (int i = 1; i <= n; i = i + 1)
    {
        s = s + i;
    }
    return s;
}
```

Exercice 9

Écrire un programme en C qui répond aux spécifications suivantes :

```c
double puissance(double x, int n)
{
    /*
    :pre-condition: x différent de zéro
    :pre-condition: n est positif
    :sortie y: double
    :post-condition: y vaut x puissance n
    */
}
```

Tableaux

Syntaxe C des tableaux

```c
void procedure_tableaux()
{
    int tab[] = {13, 8, 8, 80, -5}; // tableau de taille 5

    int taille_tab = sizeof(tab)/sizeof(tab[0]); // taille du tableau, donc 5

    int x_0 = tab[0];    // 13
    int x_max = tab[3];  // 80
    int x_min = tab[4];  // -5
}
```

Tableaux

Création

```c
int main() // nombres_n
{
    int n = 10;
    int i = 0;
    int tab[n]; // tableau de taille n
    while (i < n)
    {
        tab[i] = i + 1;
        i = i + 1;
    }
    return 0;
}
```

Lecture

```c
int somme_tab(int tab[], int n)
{
    /*
    :pre-condition: tab est de taille n
    */
    int s = 0;
    int i = 0;
    while (i < n)
    {
        s = s + tab[i];
        i = i + 1;
    }
    return s;
}
```

Exercice 10

Testez la fonction `somme_tab`

```c
int somme_tab(int tab[], int n)
{
    /*
    :pre-condition: tab est de taille n
    */
    int s = 0;
    int i = 0;
    while (i < n)
    {
        s = s + tab[i];
        i = i + 1;
    }
    return s;
}
```

Réserver de la mémoire

Adresse : emplacement en mémoire sur l'ordinateur.

Dans les systèmes à 64 bits, une adresse est généralement stockée sur 8 octets.

```
void main()
{
    // tab est une variable qui contient une adresse
    float tab[] = {1, 2, 3};
    float x = tab[0]; // La valeur est située à l'adresse pointée par "tab"
}
```

Portée d'une variable

La **portée** d'une variable est le contexte dans lequel la elle est définie.

A la fin de l'exécution d'une fonction

- les variables locales de la fonction n'existent plus
- la mémoire de ces variables est automatiquement libérée

Portée de variable : problème 1

```
void f()
{
    int y = 0;
}

void probleme_1()
{
    f();
    int x = y;
}
```

Portée de variable : problème 2

```
void probleme_2(float x)
{
    if (x < 0)
    {
        int y = 0;
    }
    int z = y;
}
```

Portée de variable : problème 3

```
TYPE_TABLEAU f()
{
    float tab[] = {1, 2, 3};
    return tab;
}

void probleme_3()
{
    TYPE_TABLEAU tab = f();

    printf("tab[0] = %f\n", tab[0]);
}
```

Pointeurs : les variables qui contiennent des adresses

Rappel : une adresse est un emplacement en mémoire sur l'ordinateur.

Syntaxe de déclaration d'un pointeur

```
type* pointeur;
```

Par exemple :

- `int*` adresse vers un type `int`
- `double*` adresse vers un type `double`
- `char*` adresse vers un type `char`

Pointeurs : récupérer l'adresse d'une variable

Syntaxe de récupération de l'adresse d'une variable ;

```
type variable;
type* pointeur = &variable;
```

Pointeurs : déclaration et accès

```
void main()
{
    int* pointeur;  // déclaration d'un pointeur
    int valeur = 10;

    // affectation du pointeur
    pointeur = &valeur;

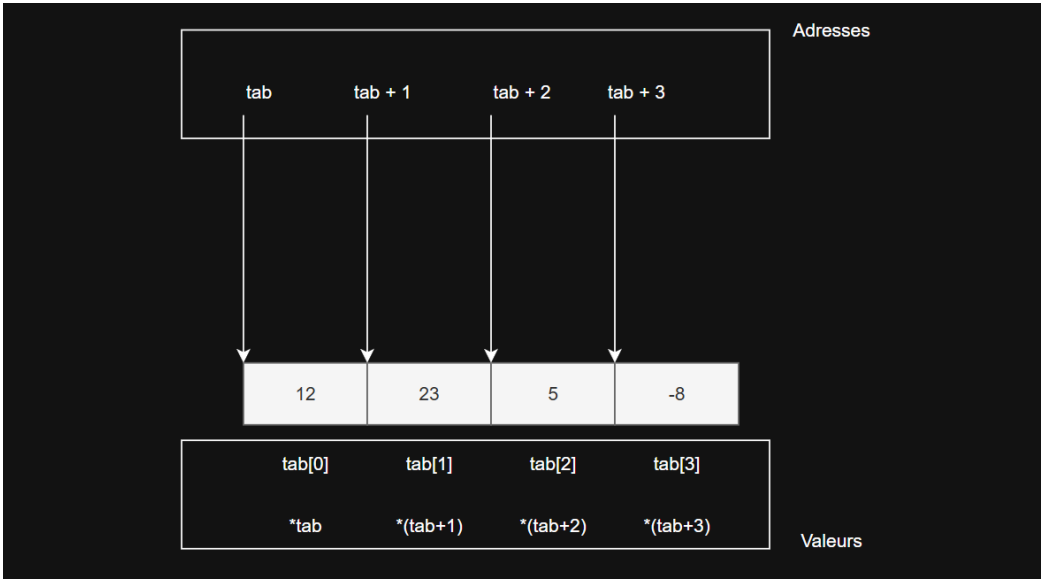
    int valeur2 = *pointeur; // accès à la valeur via l'adresse

    *pointeur = 20; // change aussi "valeur"
}
```

Pointeurs : relation avec les tableaux

Qu'est-ce qui est affiché par le code suivant ?

```
void main()
{
    // tab est une variable qui contient une adresse
    float tab[] = {1, 2, 3};
    float x = tab[0]; // x contient la valeur à l'adresse de "tab"
    float* tab_aussi = tab;
    tab_aussi[0] = 10;
    printf("tab[0] = %f\n", tab[0]);
    printf("tab[0] = %f\n", *tab);
    printf("x = %f\n", x);
}
```


Adresses
tab
tab + 1
tab + 2
tab + 3
12
23
5
-8
tab[0]
tab[1]
tab[2]
tab[3]
*tab
*(tab+1)
*(tab+2)
*(tab+3)
Valeurs

Exemple

```c
int somme_tab(int tab[], int n)
{
    /*
    :pre-condition: tab est de taille n
    */
    int s = 0;
    int i = 0;
    while (i < n)
    {
        s = s + tab[i];
        i = i + 1;
    }
    return s;
}
```

```c
int somme_tab(int* tab, int n)
{
    /*
    :pre-condition: tab est de taille n
    */
    int s = 0;
    int i = 0;
    while (i < n)
    {
        s = s + *(tab + i);
        i = i + 1;
    }
    return s;
}
```

Entrée utilisateur

On peut lire une entrée utilisateur via `scanf` dans `stdio.h` :

```
int scanf(const char * format, ...);
```

Par exemple :

```
#include <stdio.h>
int main ()
{
    char str[80];
    int i;
    printf ("Entrez votre nom de famille: ");
    scanf ("%79s",str);
    printf ("Entrez votre age: ");
    scanf ("%d",&i);
    return 0;
}
```

Annexe : fonctions utilitaires

Paramètres du programme (C)

```c
int main(int argc, char** argv)
{
    // argc : nombre d'arguments
    // char** : arguments

    char* arg1 = argv[0]; // premier argument
    char* arg2 = argv[1]; // premier argument

    // etc.. jusqu'à (argc - 1)

    return 0;
}
```

Paramètres du programme (Python)

```python
import sys

def main():
    # sys.argv (de type list[str]) : arguments
    argc = len(sys.argv)

    arg1 = sys.argv[0] # premier argument (str)
    arg2 = sys.argv[1] # deuxième argument (str)

    # etc.. jusqu'à (argc - 1)
```

Manipulation de fichiers (C)

On utilise des fonctions de `stdio`

La fonction `fopen` prend deux arguments

- le premier argument est le chemin vers le fichier
- le deuxième est le mode d'ouverture (e.g. lecture, écriture, ...)

Elle renvoie un pointeur vers une structure de manipulation du fichier :

```
FILE* fopen(char *chemin, char *mode);
```

Après utilisation, le fichier peut être fermé via `close` :

```
int fclose(FILE* flux);
```

qui renvoie le code d'erreur.

Manipulation de fichiers (C)

Les modes d'ouvertures :

- `mode = "r"` : lecture
- `mode = "w"` : écriture
- `mode = "r+"` ou `mode = "w+"` : lecture et écriture

Manipulation de fichiers (C)

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    FILE *fp = fopen("texte.txt", "r");
    if (fp == NULL)
    {
        return EXIT_FAILURE;
    }
    printf("Le fichier texte.txt existe\n");
    if (fclose(fp) == EOF)
    {
        return EXIT_FAILURE;
    }
    return 0;
}
```

Manipulation de fichiers (C)

Écriture d'un caractère :

```
int fputc(int ch, FILE *flux);
```

Lecture d'un caractère :

```
int fgetc(FILE *flux);
```

Déplacement du curseur :

```
int fseek(FILE *flux, long int decalage, int referentiel);
```

Utilisées dans TP8 et TP9 ...

Manipulation de fichiers (Python)

```python
import os

def test_fichier():
    # sans le test des erreurs
    f = open("texte.txt", "r")  # ouverture et mode lecture
    f.read()  # lecture du fichier en entier
    f.readline()  # lecture d'une seule ligne
    f.close()  # fermeture du fichier

    f = open("texte.txt", "w")  # ouverture et mode écriture
    f.write("nouvelle ligne\n") #
    f.close()  # fermeture du fichier
```

Gestion des erreurs (C)

```c
#include <stdio.h>
#include <stdlib.h>

float divide(float a, float b)
{
    if (b == 0.0f)
    {
        fprintf(stderr, "ValueError: division by zero\n");
        exit(EXIT_FAILURE);
    }
    return a / b;
}
```

Gestion des erreurs (Python)

```python
def divide(a: float, b: float) -> float:
    if b == 0:
        raise ValueError("Division par zero")
    return a / b
```

Allocation dynamique (C)

La bibliothèque `#include <stdlib.h>` permet l'allocation et libération mémoire :

- `malloc` pour l'allocation

```
int nb_valeurs = 200;
double* alloc_dynamique = malloc(nb_valeurs * sizeof(double));
```

- `free` pour la libération

```
free(alloc_dynamique);
```

Allocation dynamique (C) : malloc

Vérifier le retour de `malloc` :

```c
int main()
{
    int nb_valeurs = 200;
    double* alloc_dynamique = malloc(nb_valeurs * sizeof(double));
    if (alloc_dynamique == NULL)
    {
        // il y a une erreur
        return 1;
    }
    free(alloc_dynamique);
    return 0;
}
```

Allocation dynamique (C) : allocation et libération

```c
#include <stdlib.h>
int* creer_tableau(int n)
{
    int* T = malloc(n * sizeof(int)); // allocation
    int i = 0;
    while (i < n)
    {
        T[i] = i;
        i = i + 1;
    }
    return T;
}

void main()
{
    int* T = creer_tableau(10);
    free(T); // liberation
}
```

Introduction à “git”

Apprendre en ligne

Quelques exemples

- https ://gitmastery.me/
- https ://learngitbranching.js.org/